Stack-Disciplined Open Addressing: Exact Undo, Exact Refusal Probabilities, and Key-Lifetime Rules for Bounded-Memory Scoped Tables

Shubhi Singh and Adhyan Jain
Vellore Institute of Technology, Vellore, India

Abstract

Compilers and interpreters keep a scoped symbol table that binds names to declarations and forgets them in last-in-first-out order when a block ends; on microcontrollers and in boot loaders the table must live in one fixed buffer, refuse a request cleanly when it is full, and bound the work of every call. Open addressing is the natural structure, but its analysis covers insertions, deletions are known to be difficult, and probe limits have been set with loose Chernoff-type bounds. We ask what the stack discipline itself gives such a table. We prove that, for every non-reordering scheme, popping the newest key by clearing its cell is an exact undo, so that every insertion-only analysis transfers; that stack order is the only deletion order safe for every table, and that the safe orders of a given table are the linear extensions of a dependency relation whose size is a queue length; that the refusal probability of a probe-limited table is exactly computable for one push, for a whole fill, for an unlimited run, and, through a schedule-free bound, for a persistent set of names with bounded churn above it; and that a fresh key per table keeps the exact law valid against adversarial names. Exhaustive enumeration in exact arithmetic and a real probe-limited table agree with every formula; at load 3/4 the exact probe limit for a failure probability of 10^{-6} is 258 against 455 for the Chernoff-type rule, and the whole-fill rule is 24% smaller than the union bound. Open problems are a limit law for refusals on wide and deep scope trees and an unconditional probe-complexity separation.

Keywords – bounded memory, exact analysis, linear probing, open addressing, scoped symbol tables, stack discipline

1. Introduction

Compilers, interpreters and virtual machines for small devices are moving onto the device. Run-time systems for Arduino-class microcontrollers now execute WebAssembly with live code updates [1], and the tooling of mainstream languages treats name resolution as a subject of its own: the scheduling of name and type queries in type checkers, and their safety under parallel checking, have been characterised with scope graphs [2, 3]. Whatever the language, the same data structure sits at the centre of name resolution: a symbol table that binds identifiers to declarations, forgets the declarations of a block when the block ends, and is consulted at every use of a name. The textbook implementation is a hash table from names to their innermost declaration together with an undo stack that restores what a scope hid [4, 5].

When the program that owns the table runs on a microcontroller, in a boot loader or under a fixed allocation, three requirements appear. The table must live in *one fixed buffer*, because the allocator is absent or must not be called. It must *refuse* a request that does not fit and stay usable afterwards, because running out of memory is an expected event and not a crash. And the *work* of one call must be bounded, even when the source text is supplied by someone else. We study a table that meets these requirements, STRATA [6]: one buffer holds an open-addressing index, one packed record per visible name and the undo stack, and an operation that does not fit changes nothing. The question is what the last-in-first-out discipline of scopes, a very special deletion order, gives such a table that a general hash table does not have.

Open addressing has a long analytical tradition, from Knuth's analysis of linear probing [7] to exact distributional results [8], and recent years have been unusually productive. Iceberg hashing gives a stable table with constant-time operations at load $1 - o(1)$ and strong space, cache and failure-probability guarantees [9], and the time/space trade-off of hash tables has been determined for a large class of structures [10]. For the simple schemes used in practice, a second probe sequence makes the worst unsuccessful search of linear probing $O(\log \log n)$ almost surely at constant load [11], and quadratic probing has been analysed in a smoothed model, with a separation between Robin Hood and anti-Robin Hood orderings [12].

The combinatorics we use comes from parking functions, which describe exactly the event that every key finds a place: bijections with spanning forests and the probabilistic properties of a uniform parking function are now understood in detail [13], and the ℓ-interval variants count the cases in which every key stays within ℓ cells of its home [14, 15]. Nothing in this literature concerns the discipline of a scoped table; the related work of Section 2 makes this precise.

All of this concerns insertions, or deletions drawn at random. The deletion order of a scoped table is neither: it is last-in-first-out, and for that order the state of an open-addressing table, its safe deletion orders and its refusal probabilities have not, to our knowledge, been stated exactly. Probe limits are chosen with Chernoff-type bounds that lose a factor growing with the limit, and whether a probe-limited table refuses an honest name is usually answered by simulation. Exact finite-size laws matter here because the tables have hundreds to thousands of cells, where asymptotic formulas are not accurate.

The second shortfall concerns adversarial input. A table that must bound the work of every call has to withstand names chosen to collide, and algorithmic denial of service against hash-based structures is a demonstrated attack class: low-rate adversarial traffic reduced an Open vSwitch packet classifier built on tuple space search to about 12% of its capacity [16]. Keyed hash functions such as SipHash are the standard defence, and reduced-round variants have been cryptanalysed [17]. Whether one key

may be reused across tables, and what an adversary with a probe oracle then gains against a probe-limited table, has not been quantified.

We prove that, for every non-reordering open-addressing scheme, popping the newest key by clearing its cell is an *exact undo*, so that every insertion-only analysis of the table transfers to push/pop workloads. We characterise the deletion orders that are safe for a given table, show that stack order is the only order safe for every table, and give the exact law of the number of keys orphaned by a deletion. These statements are elementary; their value lies in what follows from them.

On this basis we derive the refusal probability of a probe-limited table exactly, for one push, for a whole fill, for an unlimited run on a fixed table and, through a schedule-free bound, for a persistent set of names with bounded churn above it. We show that a fresh key per table keeps the law exact against adversarial names while a persistent key does not, and we give an exact space ledger. Every formula is checked by exhaustive enumeration in exact arithmetic, against the real table and by an independent derivation, and all numbers are regenerated by one script. The refusal law and the limit law of the orphan count are classical as quantities and are credited as such.

Section 2 reviews fifteen recent works on hash tables, on scoped name resolution on constrained devices and on adversarial robustness, with their research gaps. Section 3 presents the proposed work: the problem, the architecture, the methods with their equations, the modules, the flow and three algorithms. Section 4 describes the implementation and analyses eight performance metrics and a comparison with related work. Section 5 concludes and Section 6 lists the references; the declarations follow. The full proofs are in the accompanying manuscript.

2. Related works

Fifteen recent works are reviewed in three groups: hash tables and open addressing (2.1–2.8), scoped name resolution and compilation on constrained devices (2.9–2.12) and adversarial robustness (2.13 –2.15). Each summary was written from the abstract of the paper and, for some, its introduction or overview, as recorded with the reference; proofs were not read, and the research gaps are our assessment relative to this work.

2.1 Bender, Kuszmaul and Kuszmaul (2021): linear probing with tombstones [18]

Solution proposed. Shows that tombstones left by deletions create an anti-clustering effect in (ordered) linear probing, so amortized insertion cost at load 1-1/x is x polylog(x) rather than x^2. Also gives 'graveyard hashing', a linear-probing variant with periodic rebuilds that plants tombstones, achieving O(x) expected cost per operation on any operation sequence.

Methodology and tools. Probabilistic analysis of ordered linear probing under insert/delete workloads with rebuild windows of size R. The analysis reduces insertion cost to crossing numbers

of intervals, then through a chain of problem transformations to a combinatorial-geometry question about monotone lattice paths (path surplus) with blue/red dots.

The bound is shown tight for hovering workloads in the classical setting ($\tilde{O}(x^{1.5}$ with an $\Omega(x^{1.5} \lg \lg x)$ lower bound); a larger rebuild window $R=n/\text{polylog}(x)$ gives $\tilde{O}(x)$. Graveyard hashing is analysed separately (the paper states it needs 5-independent hashing).

Result and effectiveness. Amortized expected insertion cost x polylog(x) at load 1-1/x with queries O(x); graveyard hashing gives O(x) per operation and 1+o(1) block transfers when x=o(B), versus the $B=\Omega(x^2$ needed earlier.

Research gap. (Our assessment) Analyses expected amortized time under random deletions with tombstones and rebuilds; it does not study LIFO (stack-ordered) deletions with exact undo, nor exact refusal probabilities for a hard probe limit.

2.2 Braverman and Kuszmaul (2024): tight analyses of linear probing [19]

Solution proposed. Settles the amortized insertion cost of linear probing with tombstones at load 1-1/x at $\Theta(x \log^{1.5} x)$, and extends the bounds from ordered linear probing to the classical unordered variant used in practice.

Methodology and tools. Reduces the problem to the path surplus problem (maximum over monotone grid paths of blue minus red Poisson dots) and gives a new analysis showing expected maximum surplus $\Theta(m \log^{0.75} m)$. A tighter reduction from path surplus back to ordered linear probing then yields matching bounds with an optimally chosen rebuild window.

Result and effectiveness. $\Theta(x \log^{1.5} x)$ worst-case amortized insertion/deletion time with O(x) queries; path surplus $\Theta(m \log^{0.75} m)$.

Research gap. (Our assessment) Asymptotic amortized cost under random (not stack-ordered) deletes with rebuilds; no exact probabilities of refusal under a fixed probe limit.

2.3 Farach-Colton, Krapivin and Kuszmaul (2024): elastic and funnel hashing [20]

Solution proposed. Gives 'elastic hashing', a non-greedy open-addressed table without reordering with O(1) amortized and $O(\log \delta^{-1}$ worst-case expected probe complexity, and 'funnel hashing', a greedy scheme with $O(\log^2 \delta^{-1}$ worst-case expected probes. This disproves Yao's 1985 conjecture that uniform probing is optimal among greedy schemes.

Methodology and tools. Constructions partition the array into geometrically organized sub-arrays (levels) so that insertions can probe far down the probe sequence before settling, bypassing the coupon-collector bottleneck. Matching lower bounds are proved: $\Omega(\log \delta^{-1}$ for any non-reordering table, $\Omega(\log^2 \delta^{-1}$ for greedy schemes, and $\Omega(\log^2 \delta^{-1} + \log \log n)$ w.h.p. (details of proofs not read).

Result and effectiveness. Insertion-only open addressing without reordering is solved up to constants: amortized O(1), worst-case expected O(log δ^{-1}; funnel hashing beats uniform probing's $\Theta(\delta^{-1}$ for greedy schemes, all with matching lower bounds.

Research gap. (Our assessment) Insertion-only model and δ-fraction slack; no deletions, no stack discipline, no hard probe cap with refusals, and the structures are far from the plain linear probing used in compiler symbol tables.

2.4 Krapivin, Kuszmaul and Wang (2026): interlinear probing [21]

Solution proposed. Introduces interlinear probing, a non-greedy insertion rule for linear probing (an insertion may skip a free slot to leave it for later) with no reordering. Given x in advance, worst-case expected insertion and positive-query time is O(x log x) at load 1-1/x, versus $\Theta(x^2$ for greedy linear probing.

Methodology and tools. Upper bounds: a stable non-greedy variant of linear probing, extended with a new stopping rule so negative queries cost $O(x \log^2 x)$, and a further extension giving O(x) amortized insertion. Lower bound: a multiscale potential argument over power-of-two block sizes compares imbalance from random home locations with the final-placement imbalance, showing Ω(x sqrt(log x)) for any stable strategy.

Upper bounds assume $x <= n^{0.49}$ and x known in advance.

Result and effectiveness. Optimal worst-case expected insertion time among stable linear-probing strategies is x (log x)^Θ(1): O(x log x) upper bound versus Ω(x sqrt(log x)) lower bound.

Research gap. (Our assessment) Insertion-only (and the 'no reordering' stable model) with expected time; no deletions, no hard probe cap with refusal probabilities, and no stack-ordered undo.

2.5 Alharbi, Banderier and Bouillaguet (2026): bounded linear probing [22]

Solution proposed. Introduces bounded linear probing: an insertion probes at most p consecutive cells and is discarded (counted as failed) if all are occupied. It interpolates between the coupon collector (p=1) and classical linear probing (p>=n). Derives the expected number of insertion attempts needed to fill the table.

Methodology and tools. Analytic combinatorics: tables are grouped by circular symmetry (as in Knuth) into full and almost-full tables with a 'trash bin' of failed insertions. A non-linear PDE for the bivariate generating functions (Theorem 1 and 2) is derived using a full algebra of operators, then solved in closed form (Theorems 3 and 4).

Asymptotics of the expected filling time are obtained from the closed forms (Theorem 5), and compared to experiments. Motivated by long-message second-preimage attacks on Merkle-Damgard hash functions.

Result and effectiveness. Closed-form generating functions for full and almost-full tables for any p, and the asymptotic expected number of insertion trials to fill a table, which decreases by almost a factor p as p grows.

Research gap. (Our assessment) The closest prior work on probe-limited linear probing, but for insertions in which a failed key is discarded and redrawn until the table is full; it has no deletions, no stack discipline, and no refuse-and-stop event with a refusal probability at a fixed load.

2.6 Bender, Kuszmaul and Zhou (2025): non-oblivious open addressing [23]

Solution proposed. Constructs a non-oblivious open-addressed hash table ('partner hashing') with O(1)-time operations even when the table is completely full, including a dynamically resized version using exactly as many slots as elements. Time bounds hold with high probability using only O(1) O(1)-wise independent hash functions.

Methodology and tools. The main technique encodes metadata in the relative ordering of the stored elements, simulating a $\Theta(n \log n)$-bit RAM with $O(\log n)$-bit words inside the permutation of elements, with O(1) access and efficient updates. The intro presents a warm-up structure first and then refinements for resizing and high-probability bounds (read at intro level only; proofs not read).

Result and effectiveness. Constant-time insert/delete/query at load factor 1 (full table), with high probability; this bypasses the $\Omega(\log \log \varepsilon^{-1}$ lower bound that holds for oblivious open addressing.

Research gap. (Our assessment) Gives up the simple structure of classical open addressing (probe sequence plus naive layout) by encoding data in element order; says nothing about plain linear probing with a probe limit, stack-ordered deletes, or exact refusal probabilities.

2.7 Bender et al. (2023): tiny pointers [24]

Solution proposed. Introduces tiny pointers: o(log n)-bit replacements for log n-bit pointers at constant-factor time overhead, with optimal fixed-size and variable-size constructions. Applies them to data retrieval, succinct dynamic BSTs, stable dictionaries, variable-size-key dictionaries and the stash problem.

Methodology and tools. Based on the abstract only: the constructions revisit classic balls-and-bins problems, and the tiny-pointer theory gives optimal sizes $\Theta(\log \log \log n + \log k)$ bits (fixed) and $\Theta(\log k)$ expected bits (variable) at load 1-1/k. Technique details were not read beyond the abstract.

Result and effectiveness. Optimal tiny-pointer sizes at load factor 1-1/k, and space-efficient solutions to five classic data-structure problems 'for free'.

Research gap. (Our assessment) Space-efficiency theory for pointer replacement; not about probe-limited linear probing, refusals, or LIFO deletion. Cite as background on space-time tradeoffs in hashing.

2.8 Aguilar-Fraga et al. (2024): ℓ-interval parking functions [14]

Solution proposed. Generalizes interval parking functions to n cars and m>=n spots (interval rational parking functions) and counts them, then studies the subset where each car parks at most l spots from its preference (l-interval).

Methodology and tools. Enumerative combinatorics: a counting formula for interval rational parking functions, recursive formulas for l-interval ones, and nondecreasing l-interval counts via the outcome map; for l=1 a bijection with barred preferential arrangements. Beyond the abstract, details were not read.

Result and effectiveness. Enumeration formulas (closed and recursive), a Fibonacci-type count for the intersection with Fubini rankings, and the l=1 bijection with barred preferential arrangements (recovering the unit-interval result of Hadaway and Harris).

Research gap. (Our assessment) The l-interval condition is a probe limit in parking-function language, but the paper counts configurations; it has no hashing interpretation, no deletion/undo, and no probability law for refusals with random keys.

2.9 van Antwerpen et al. (2018): scopes as types [25]

Solution proposed. Treats scopes as types so scope graphs can model the internal structure of non-simple types (structural records, generic classes). Extends scope graphs with scoped relations and queries, and introduces Statix, a meta-language for static semantics based on scope graphs plus constraints.

Methodology and tools. Abstract only (full text not read). The approach extends scope graphs with scoped relations and generalized queries and is realised in the Statix DSL, which specifies static semantics via scope graphs and constraints.

Evaluation, per the abstract, is by case studies: simply-typed lambda calculus with records, System F, and Featherweight Generic Java.

Result and effectiveness. Abstract reports only that the approach and Statix design were evaluated in the three case studies; no quantitative figures are stated in the abstract.

Research gap. Our assessment: Scope graphs are a declarative, unbounded-memory model of name binding and resolution; it does not address a fixed byte buffer, LIFO push/pop of a hash table, probe limits, or the probability that an insertion is refused.

2.10 Zwaan et al. (2022): incremental type checking from scope graphs [26]

Solution proposed. Derives incremental type checkers automatically from Statix specifications: name-resolution queries in scope graphs define dependencies between compilation units, and a query-confirmation algorithm finds queries whose answers changed after an edit so only those units are re-analysed.

Methodology and tools. Abstract only. Technique: dependency derivation from scope-graph queries, a query-confirmation algorithm, splitting checking into context-free and context-sensitive parts, and a generic mechanism for mutual dependencies.

Evaluation compares against non-incremental parallel execution on synthetic benchmarks and real-world projects.

Result and effectiveness. Abstract reports speedups up to 147x on synthetic benchmarks and up to 21x on real-world projects over non-incremental parallel execution, with initial overhead below 10%.

Research gap. Our assessment: Incrementality is about re-checking after edits with unbounded memory; it does not cover bounded-memory tables, deterministic undo on scope exit, or refusal-probability guarantees.

2.11 Ball, de Halleux and Moskal (2019): Static TypeScript [27]

Solution proposed. Static TypeScript (STS), a TypeScript subset with a compiler/linker toolchain written in TypeScript running in the browser. Programs compile to ARM Thumb machine code linked against a precompiled C++ runtime, aimed at Cortex-M devices with 16-256 kB RAM (e.g. micro:bit, 16 kB).

Methodology and tools. Read in the draft: description of the STS language subset (static layout of classes, nominal-style class compilation), the compilation scheme, and the MakeCode deployment context.

Evaluation compares STS (native Thumb and a VM) with C (gcc), Duktape, IoT.js, MicroPython, CPython and Node.js on STM32F401RE (96 kB RAM), ATSAMD51G19 (192 kB RAM) and a Raspberry Pi Zero (reference only), using small benchmarks including binary trees, richards, fann and nbody.

Result and effectiveness. Draft states STS is over an order of magnitude faster than embedded interpreters on property-heavy benchmarks, less than 2x slower than Node.js there, interpreters run out of memory sooner, and STS runs on devices with as little as 16 kB RAM.

Research gap. Our assessment: Shows full compilation (including scoping and typing) targeting tiny devices but runs the compiler in a browser on the host; it does not give a fixed-buffer on-device symbol table or probabilistic analysis of its capacity limits.

2.12 Mochizuki and Yamazaki (2025): BlueScript [28]

Solution proposed. Proposes a 'disaggregated' VM that offloads as many VM components as possible (incl. incremental and dynamic compilers) to a host, using a 'shadow machine' on the host that mirrors microcontroller execution state to cut communication overhead; instantiated as the BlueScript VM.

Methodology and tools. Abstract only. Design and implementation of the BlueScript VM with host-side offloaded components and a shadow-machine data structure.

Experiments compare against MicroPython and Espruino (abstract level; setup details not read).

Result and effectiveness. Abstract reports offloading does not seriously compromise expected benefits; the offloaded incremental compiler gives faster execution than MicroPython and Espruino with interactivity comparable to MicroPython, and the offloaded dynamic compiler improves VM performance.

Research gap. Our assessment: Argues for moving memory-hungry compiler state off the device; leaves open the case where scoped symbol tables must live on the device itself in one fixed buffer.

2.13 Naor and Yogev (2019): Bloom filters in adversarial environments [29]

Solution proposed. Defines an adversarial model in which an adversary picks inputs and queries adaptively from earlier answers, and relates secure Bloom filters to cryptography. Gives a construction against query-bounded unbounded adversaries.

Methodology and tools. Abstract only. Cryptographic reductions and information-theoretic constructions for computationally bounded and query-bounded unbounded adversaries.

Result and effectiveness. Per abstract: non-trivial Bloom filters against efficient adversaries exist iff one-way functions exist; against t queries a filter with $O(n \log(1/\varepsilon) + t)$ bits exists, versus $n \log(1/\varepsilon)$ for non-adaptive adversaries.

Research gap. Our assessment: sets the adaptive-adversary framework we can borrow, but treats approximate membership, not open-addressing tables; no exact finite-size refusal law and no deletion model.

2.14 Clayton, Patton and Shrimpton (2019): probabilistic data structures in adversarial environments [30]

Solution proposed. Gives a provable-security treatment of probabilistic data structures when data and queries may depend on internal randomness. Shows that salts or secret keys yield provable security with little overhead.

Methodology and tools. Abstract only. A syntax covering many structures and security notions that support error bounds under powerful attacks; focuses on Bloom filters, also counting Bloom filters and count-min sketch.

Result and effectiveness. Per abstract: findings for the traditional (unkeyed) structures are largely negative; simple keyed or salted variants give provable security at little overhead.

Research gap. Our assessment: close in spirit to our key-lifetime question, but for approximate-membership/sketch structures; does not give exact probe-limit refusal probabilities or handle stack-ordered undo.

2.15 Blair et al. (2020): HotFuzz [31]

Solution proposed. HotFuzz is a framework that finds algorithmic-complexity (AC) vulnerabilities in Java libraries by micro-fuzzing individual methods. A genetic algorithm evolves Java objects to trigger worst-case performance. Confirmed cases are replayed as standalone Java programs.

Methodology and tools. Abstract only. Micro-fuzzing with a genetic algorithm over Java objects; seed inputs derived by Small Recursive Instantiation (SRI); candidate witnesses are synthesized into Java programs and monitored for high CPU utilization. Evaluated on the JRE, the 100 most popular Maven libraries and DARPA STAC challenges, comparing SRI seeds against empty-value seeds.

Result and effectiveness. Per abstract: verified known AC vulnerabilities, found previously unknown ones reported to vendors (confirmed by IBM and Oracle), and SRI seeds outperformed empty seeds. Abstract gives no counts.

Research gap. Our assessment: finds implementation-level worst-case inputs but gives no exact probability law; it does not treat keyed hashing, probe limits or stack-ordered deletion in a hash table.

3. Proposed work/system

The work is a theory of what the last-in-first-out discipline of scopes gives an open-addressing index. The statements below follow the numbering of the full manuscript, which contains every proof and the programs that check them; the text here explains the system, the method and the results.

3.1 Problem formulation

Let a table consist of m cells arranged on a ring. A key x has a home $h(x)$ in $\{0,\ldots,m-1\}$, drawn independently and uniformly (the random-function model for a keyed hash function), and an insertion rule places x in the first empty cell found by probing $h(x)$, $h(x)+1$, and so on. A rule is *non-reordering* if a key never moves after it is placed and the cell chosen for an insertion depends only on the set of occupied cells and on the key's own probe sequence. A scoped symbol table receives an oblivious schedule of *push* (declare a name) and *pop* (leave a scope) operations; pops remove the newest live keys, so the live keys form a stack. A *probe limit* L makes the table refuse a push whose first L probed cells are all occupied, and a refusal changes nothing.

Four questions follow. **Q1 (state):** is the table after a pop distributed like an insertion-only table? **Q2 (order):** which orders of clearing cells keep every lookup correct, and is stack order special? **Q3 (refusal):** how large must L be so that an honest push is refused with probability at most ε, for one push, for a whole fill, for an unlimited run on a fixed table, and for a persistent set of names with bounded churn above it? **Q4 (keys):** what changes when an adversary can query the hash? The answers have to be exact at finite m, because the tables of interest have a few hundred to a few thousand cells, where asymptotic formulas are not accurate.

3.2 System architecture

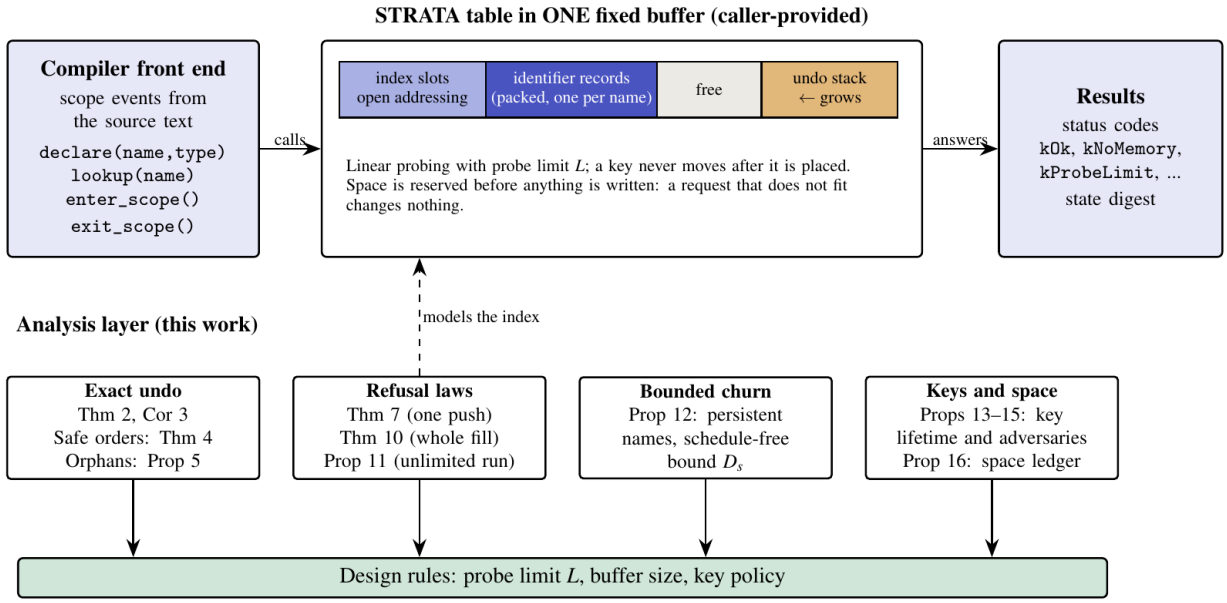


Figure 1. Architecture. The compiler front end issues four calls to a STRATA table that keeps its open-addressing index, packed identifier records and undo stack in one fixed buffer. The analysis layer models the index and produces design rules for the probe limit, the buffer size and the key policy.

Figure 1 shows the system. The compiler front end issues four calls: declare, lookup, enter scope and exit scope. STRATA keeps the index, one packed record per visible name and the undo stack of the open scopes in one caller-provided buffer. A declaration that hides an outer name saves the old record on the undo stack, and leaving a scope pops the entries in reverse order. Space for the record and the undo entry is reserved before anything is written, so a request that does not fit returns `kNoMemory` and leaves the table unchanged; a request whose probe sequence meets L occupied cells returns `kProbeLimit`, also without change.

The analysis layer below the table has four parts that mirror the four questions: exact undo and deletion orders (Theorems 2 and 4, Proposition 5), the refusal laws (Theorems 7 and 10, Propositions 11 and 12), the key-lifetime results (Propositions 13–15) and the space ledger (Proposition 16). The dashed arrow in Figure 1 marks the modelling step: the index of the table is the ring of cells, and everything proved about the ring is a statement about that index. The design rules at the bottom are the product: a probe limit, a buffer size and a key policy chosen from exact numbers instead of loose bounds.

3.3 Detailed methodology

3.3.1 Exact undo

Write Ins(a_1,…,a_n) for the table obtained by inserting the keys in this order into an empty table. **Theorem 2 (undo).** For every non-reordering rule, after any sequence of pushes and pops the table at time t is the insertion-only table of the live keys, in the order in which they were pushed:

$$T_t = \mathrm{Ins}\left(a_1, \ldots, a_{n_t}\right) \tag{1}$$

The proof is by induction. An insertion changes exactly one empty cell, chosen from the set of occupied cells alone, so clearing that cell restores the table that existed before the insertion (Figure 2). Consequently the table has, at every moment, the law of an insertion-only table, and every insertion-only result about the table's state or the cost of the next operation transfers to push/pop workloads, up to the first refusal (Corollary 3).

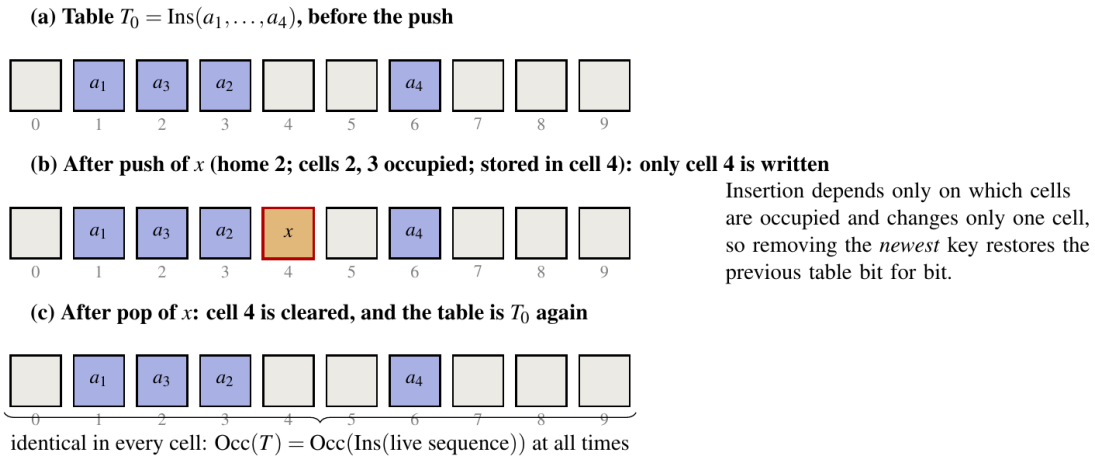


Figure 2. Exact undo. Pushing *x* writes one empty cell; popping *x* clears it, and the table is identical in every cell to the table before the push.

3.3.2 Safe deletion orders and the orphan law

Let *c*(*x*) be the cell of key *x*. A key *y depends on x*, written $y \lhd x$, if the cell of *x* lies strictly before the cell of *y* on the probe path of *y*:

$$y \lhd x \;\Leftrightarrow\; c(x) \in \{h(y),\ h(y)+1,\ \ldots,\ c(y)-1\} \tag{2}$$

Theorem 4. (a) For a given table, the orders of clearing cells that keep every lookup correct are exactly the linear extensions of $\lhd$: every key is cleared before the keys it depends on. (b) Reverse insertion order, that is stack order, is the only order that is safe for every hash assignment, for greedy schemes with probe sequences that start anywhere and have a second distinct cell. The newest key has no dependents, because a key's path crosses only cells that were filled before it (Figure 3).

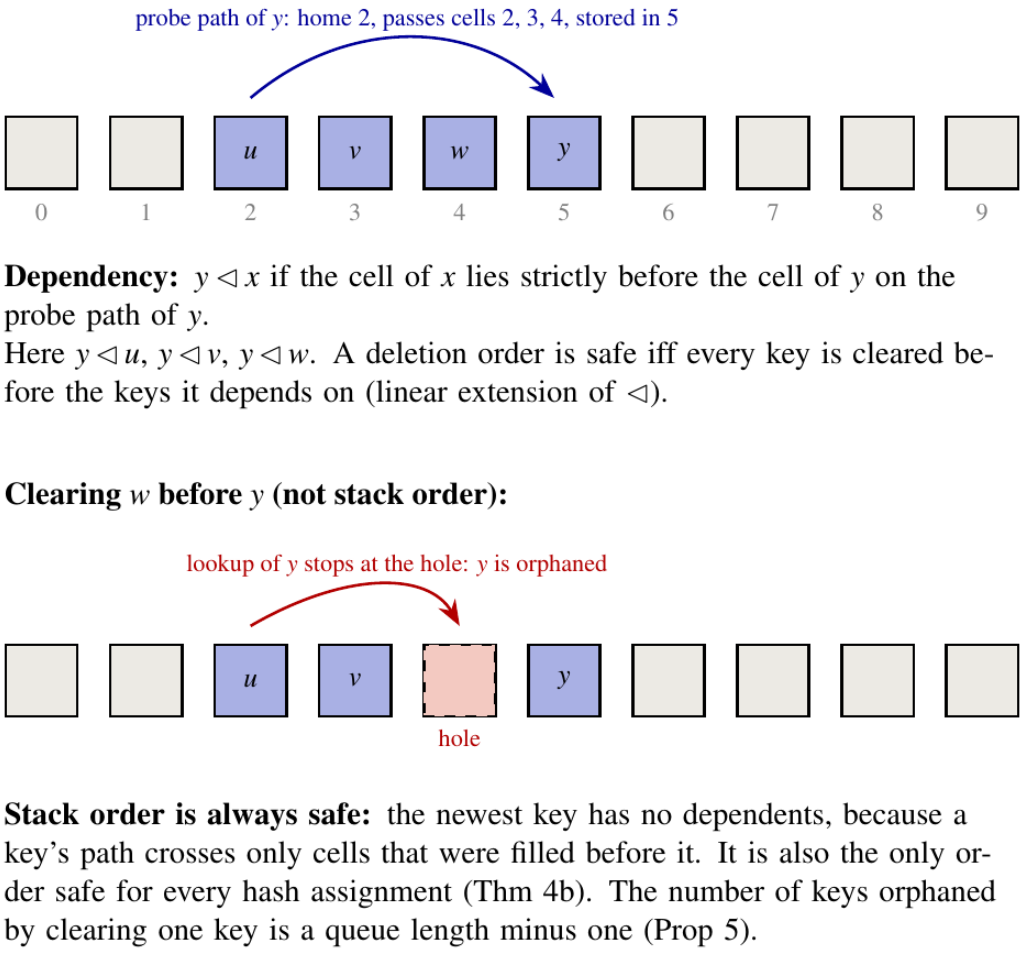


Figure 3. Dependency and orphans. Key *y* probed cells 2, 3 and 4 before it was stored in cell 5, so it depends on *u*, *v* and *w*. Clearing *w* first leaves a hole on its path and orphans *y*.

The number of keys that depend on a key is the length of a queue minus one. Linear probing is a queue: cell c receives A_c keys whose home it is, one key is served per cell, and the queue length Y_c obeys a Lindley recursion. With $\alpha = n/m$ the mean number of dependents and the limiting fraction of keys with none are

$$\operatorname{orph}(x) = Y_c - 1, \quad Y_c = \left(Y_{c-1} - 1\right)^+ + A_c \tag{3}$$

$$\mathbb{E}[\operatorname{orph}] = \frac{\alpha}{2(1-\alpha)}, \quad \mathbb{P}(\operatorname{orph} = 0) \to \frac{(1-\alpha)(e^{\alpha}-1)}{\alpha} \tag{4}$$

At load 1/2 the mean is 0.5 and 65% of keys have no dependents; at load 3/4 the mean is 1.5 and only 37% have none. Out-of-order deletion is therefore a gamble that gets worse as the table fills. The queue representation of linear probing and the limit law are due to Janson [32], and the single-deletion form of the safe-order test to Sanders [33]; the exact finite-size law, the reading as orphans, the characterisation of safe orders and the converse are new as stated.

3.3.3 The exact refusal law

Let $n = m - k$ keys sit on the ring with independent uniform homes. **Theorem 7.** The probability that a new key with an independent uniform home finds its first L cells occupied is

$$R_m(n,L)=\frac{k}{m}\sum_{g=L+1}^{n+1}(g-L)\ \pi_g,\ \ \pi_g=\frac{B(g)\ Q_{k-1}(m-g)}{Q_k(m)} \tag{5}$$

$$B(g)=\frac{g^{g-1}}{g!},\ \ Q_j(x)=\frac{j\ x^{x-j-1}}{(x-j)!} \tag{6}$$

The proof cuts the ring at its k empty cells (Figure 4). Each empty cell and the run of occupied cells after it form a block whose size is Borel distributed, by the cycle lemma, and the blocks, conditioned on filling the ring, are exchangeable. A new key is refused exactly when its home starts a window of L occupied cells, and a block of g cells contains $(g - L)^+$ such homes; summing gives Equation (5). The tail is of order $\rho^L\ L^{-3/2}$ with $\rho = \alpha\ e^{1-\alpha}$, so the Chernoff-type bound used for probe limits is loose by a factor that grows like $L^{3/2}$. The law is classical as a quantity (it is the tail of the unsuccessful-search cost, Knuth [7], Janson and Viola [8]); what is new is the elementary derivation, the explicit finite-size sum and its use for design.

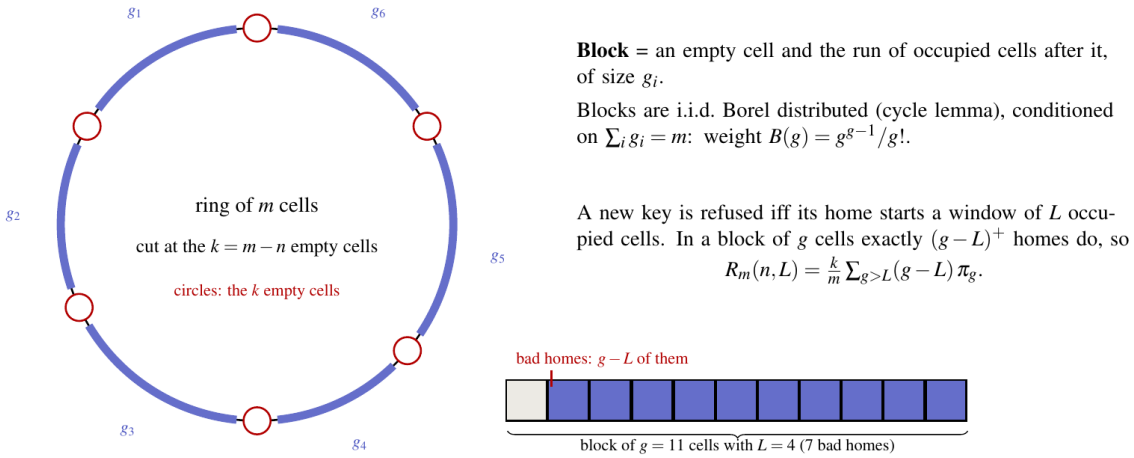


Figure 4. Block decomposition used to prove the refusal law. The ring is cut at its k empty cells; a block is an empty cell and the run after it. A window of L occupied cells inside a block of g cells has $g - L$ bad homes.

3.3.4 Whole fill and unlimited run

Two further questions have exact answers. For a monotone fill of n pushes, the probability that no push is refused is the probability that the maximum displacement stays below L; the counts are those of the ℓ-interval parking functions [14, 15] and have a closed generating function (Theorem 10). For a table that holds n keys and is then used for any number of push/pop pairs of fresh names, the table returns to the same state after every pop (Theorem 2), so the pushes are independent given the table, and no push is ever refused exactly when every block has at most L cells (Proposition 11):

$$\mathbb{P}(W=0)=\frac{[z^m]\left(\sum_{g=1}^{L}B(g)z^g\right)^k}{[z^m]\left(\sum_{g\geq 1}B(g)z^g\right)^k} \tag{7}$$

$$P_{\text{fill}}=\frac{n!}{m^n}\cdot\frac{m}{k}\ [z^n]\ \Phi_L(z)^k \tag{8}$$

The large-size behaviour of both events is that of the maximum displacement of linear probing, studied by Pittel and by Petersson [34, 35]; the exact finite-size forms and the design values are ours.

3.3.5 Persistent names with bounded churn

A real program has global names that are never popped and local scopes that come and go. Call the global names the *trunk* and let at most s further names be live above it, counting the one being pushed. Call a table *s-vulnerable* if some window of L consecutive cells contains at most $s - 1$ empty cells.

Proposition 12. (a) Any refusal in any such schedule, however long, implies that the trunk is s-vulnerable. (b) This is the supremum: a chain of at most s pushes fills the empty cells of the window and pushes at its start, which is refused, so the refusal probability after N fresh chains increases to the vulnerability probability. (c) Cutting the ring at its empty cells, vulnerability is the event that some s consecutive gaps sum to at least $L - s + 1$, which gives a closed-form bound (D_s):

$$V_s = \mathbb{P}\left(\exists\ i:\ g_i + g_{i+1} + \ldots + g_{i+s-1} \geq L - s + 1\right) \leq D_s = \min\left(1,\ k\ \Pi_s\right) \tag{9}$$

$$\mathbb{P}\left(\text{refusal after } N \text{ chains}\right) \geq V_s\left(1 - e^{-N/m^s}\right) \tag{10}$$

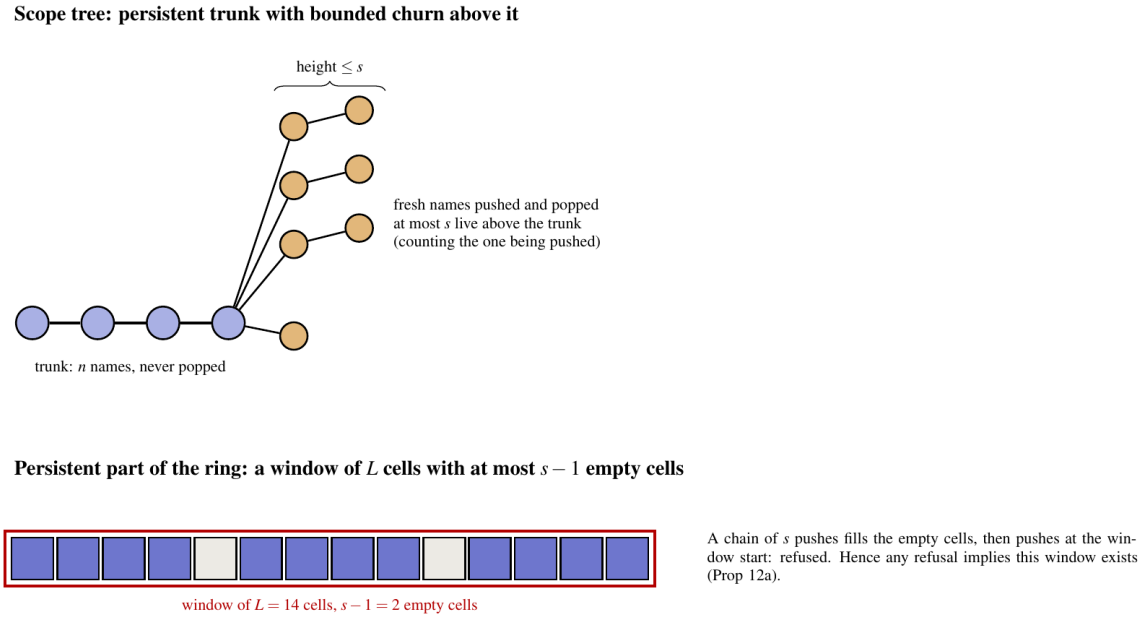


Figure 5. Persistent trunk with bounded churn. Scope trees whose branches above the persistent names have height at most s; a refusal needs a window of L cells with at most $s - 1$ empty cells in the persistent part.

Figure 5 shows the situation. The bound depends on neither the length of the run nor the order of the calls, so it is a guarantee that holds for the whole life of the table. It is conservative for short runs and uninformative for schedules that swing over a wide band; a limit law for such scope trees remains open.

3.3.6 Probe complexity, keys and space

Because the table is the insertion-only table of its live keys (Theorem 2), the worst-case and the amortised cost of a push under any oblivious stack-disciplined workload both equal the worst-case cost of the insertion-only table (Theorem 6):

$$\text{cost} = \max_{j \le (1-\delta)m} q_j \tag{11}$$

Combined with the published insertion bounds [20, 21], the optimum under stack discipline is $\Theta(\log 1/\delta)$ for non-greedy and $\Theta(\log^2 1/\delta)$ for greedy schemes, strictly between the insertion-only amortised optimum and the cost of random deletions. For keys, an independent key per table keeps the exact law valid against arbitrary adversarial names (Proposition 13); with one persistent key and a probe oracle, after about m ln m queries an adversary knows a name for every cell and can build a run of occupied cells that exceeds the law (Propositions 14 and 15). The space ledger (Proposition 16) gives exact identities for the words used by five single-buffer layouts at every instant.

3.4 Modules and their functions

Table 1. Modules of the proposed work, the result each one provides and the program that checks it.

Module	Function	Result	Checked by
Exact undo	pop = clear one cell; table = insertion of the live keys	Thm 2, Cor 3	lifo_check.py
Dependency and orphans	safe deletion orders; orphan law	Thm 4, Prop 5	orphan_law.py
Refusal law	exact $R_m(n, L)$, tail, probe-limit design	Thm 7, Cor 8	exact_law.py, design_table.py, law_vs_table.cpp
Fill and unlimited run	whole-fill and unlimited-run probabilities	Thm 10, Prop 11	fill_law.py, sustain_law.py
Bounded churn	schedule-free bound D_s	Prop 12	band_law.py
Probe complexity	cost under stack discipline	Thm 6	derivation; published bounds
Key lifetime	fresh versus persistent key	Props 13–15	adaptive_key_demo.cpp
Space ledger	exact space identities	Prop 16	space_model.py, proof_checks.py
Interactive page	calculators and models of the results	all of the above	npm test (33 checks)

3.5 Flow of the operations

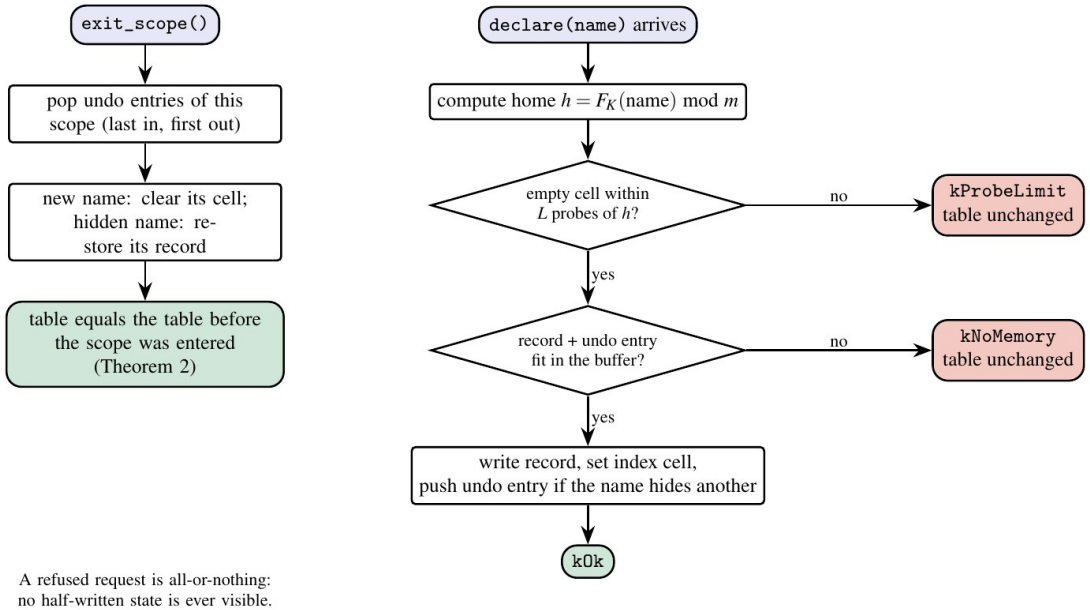


Figure 6. Flow of a declaration and of a scope exit. A declaration is refused, with the table unchanged, when no empty cell is found within L probes or when the record and its undo entry do not fit; a scope exit pops the undo entries in reverse order.

Figure 6 gives the flow. A declaration computes the home with a keyed hash and probes at most L cells. If no empty cell is found, the table answers `kProbeLimit`; if the record and the undo entry do not fit in the buffer, it answers `kNoMemory`; in both cases nothing has been written. Otherwise it writes the record, sets the index cell and, if the name hides an outer one, pushes an undo entry. Leaving a scope pops the undo entries of that scope in reverse order: a new name has its cell cleared, a hidden name has its record restored. By Theorem 2 the table then equals the table that existed before the scope was entered.

The two exits of the flow are what the analysis quantifies. `kProbeLimit` is the event whose probability the refusal laws compute; `kNoMemory` is governed by the space ledger. Both are all-or-nothing, which is what makes the exact undo meaningful: a refused push never leaves a half-written state to undo.

3.6 Algorithms

Algorithm 1. Push with probe limit and scope exit (all-or-nothing)

1: **procedure** PUSH(name)
2: $h \leftarrow F_K$(name) mod m; $c \leftarrow h$
3: **for** $t = 0$ **to** $L - 1$ **do** **if** cell c is empty **then break**; $c \leftarrow (c + 1) \bmod m$
4: **if** no empty cell was found **then return** kProbeLimit ▷ table unchanged
5: **if** the record and the undo entry do not fit **then return** kNoMemory ▷ table unchanged
6: write the record; set the index cell c; **if** name hides an outer name **then** push the saved record on the undo stack
7: **return** kOk
8: **procedure** EXIT_SCOPE()
9: **for each** undo entry of this scope, newest first **do**
10: **if** the entry marks a new name **then** clear its index cell **else** restore the saved record

Algorithm 1 is the table's behaviour. Because c is decided by the occupied cells alone and EXIT_SCOPE clears exactly the cells that PUSH wrote, Theorem 2 applies.

Algorithm 2. Design rule: smallest probe limit for a target failure probability

1: **input** m, n, target ε, and the event $\in$ {one push, unlimited run, churn depth s}
2: **output** the smallest L whose failure probability is at most ε
3: $lo \leftarrow 2$; $hi \leftarrow m$
4: **while** $lo < hi$ **do**
5: $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$
6: **if** FAIL(mid) $\le \varepsilon$ **then** $hi \leftarrow mid$ **else** $lo \leftarrow mid + 1$
7: **return** lo ▷ FAIL is R_m(n, L) of Eq. (5), P(W>0) of Eq. (7) or D_s of Eq. (9)

Algorithm 2 turns the exact laws into design values; FAIL is decreasing in L, so bisection applies. For D_s the five values below the answer are also checked.

Algorithm 3. Test for *s*-vulnerability and the refusing chain

input occupancy O of the persistent names, probe limit L, churn depth s
  compute $e(i)$ = number of empty cells in the window of L cells starting at cell i, for all i (sliding sum)
  $i^* \leftarrow$ the start of a window with the fewest empty cells
  if $e(i^*) > s - 1$ **then return** "not s-vulnerable: no such schedule can be refused"
  for each empty cell c in the window **do** push a name whose home is c ▷ it is stored in c
  push a name whose home is i^* ▷ all L cells from its home are occupied: refused
  return the chain (at most s pushes)

Algorithm 3 is the constructive half of Proposition 12(b) and is what the interactive page replays.

Algorithm 1 is the behaviour of the table, Algorithm 2 is the way the laws are used, and Algorithm 3 is the proof of the sharpness of the churn bound turned into a procedure. All three are implemented: the first in the C++ library, the second in the Python programs that generate the tables of Section 4, and the third in the verification program for Proposition 12 and in the interactive page.

Contribution of this work. (C1) Exact undo and its converse: safe deletion orders and the orphan law. (C2) Exact refusal probabilities for probe-limited tables: one push, a whole fill, an unlimited run, and bounded churn above a persistent trunk. (C3) Key-lifetime rules against adaptive adversaries. (C4) An exact space ledger. The refusal law and the orphan limit law are classical as quantities and are credited as such; the characterisation of safe orders, the exact finite-size forms, the use of the laws for design and the churn bound are the contribution.

4. Implementation and result analysis

4.1 Tools, configuration and commands

All laws are implemented twice and checked against each other and against the real table. The table itself is the header-only C++14 library of the STRATA repository [6], compiled with `g++ -std=c++14 -O2` [36]. The exact formulas, the enumerations and the design tables are Python 3 programs that use exact rational arithmetic (the `fractions` module) for every check on small rings and floating point with logarithms for the large-size tables; the figures are drawn with LaTeX (TikZ and PGFPlots). One script, `bash theory/verify/run_all.sh`, runs every program, stops on the first mismatch and rebuilds the tables, figures and the manuscript; it takes about ten minutes on a laptop-class machine. The interactive page is a React [37] application built with Vite [38]; `npm start` serves it and `npm test` runs 33 checks, among them every formula against the Python reference values and the model's claims by brute force.

Two further components were used for validation. A C++ program drives the real probe-limited table, with a keyed hash and a random key per trial, and compares the observed frequency of a refused push with the exact law (400 000 trials per setting) and compares a table with the probe limit set against an unlimited one in lock step. The repository's earlier simulation of fresh insertions into tables of 65 536 slots (10 million samples per row) provides an independent empirical reference for the law.

Figure 7 summarises the verification method, and Figure 8 shows the interactive page used to explore the results.

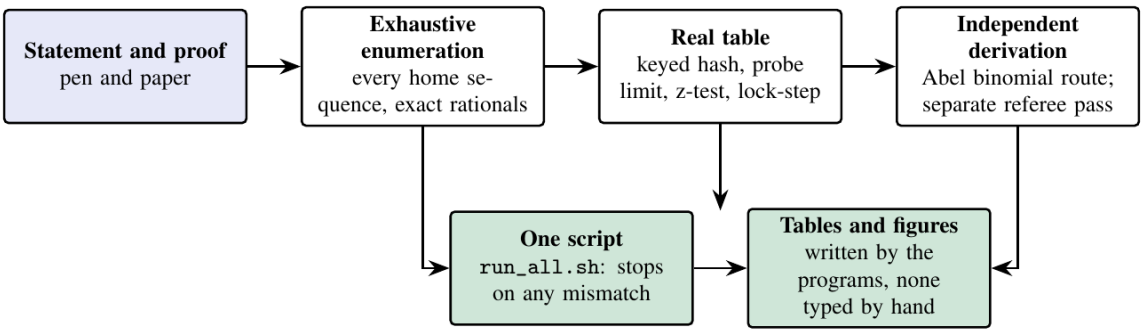


Figure 7. Verification method. Each result is proved on paper, checked by exhaustive enumeration in exact arithmetic, compared with the real table, re-derived independently, and every number in the tables is written by the programs.

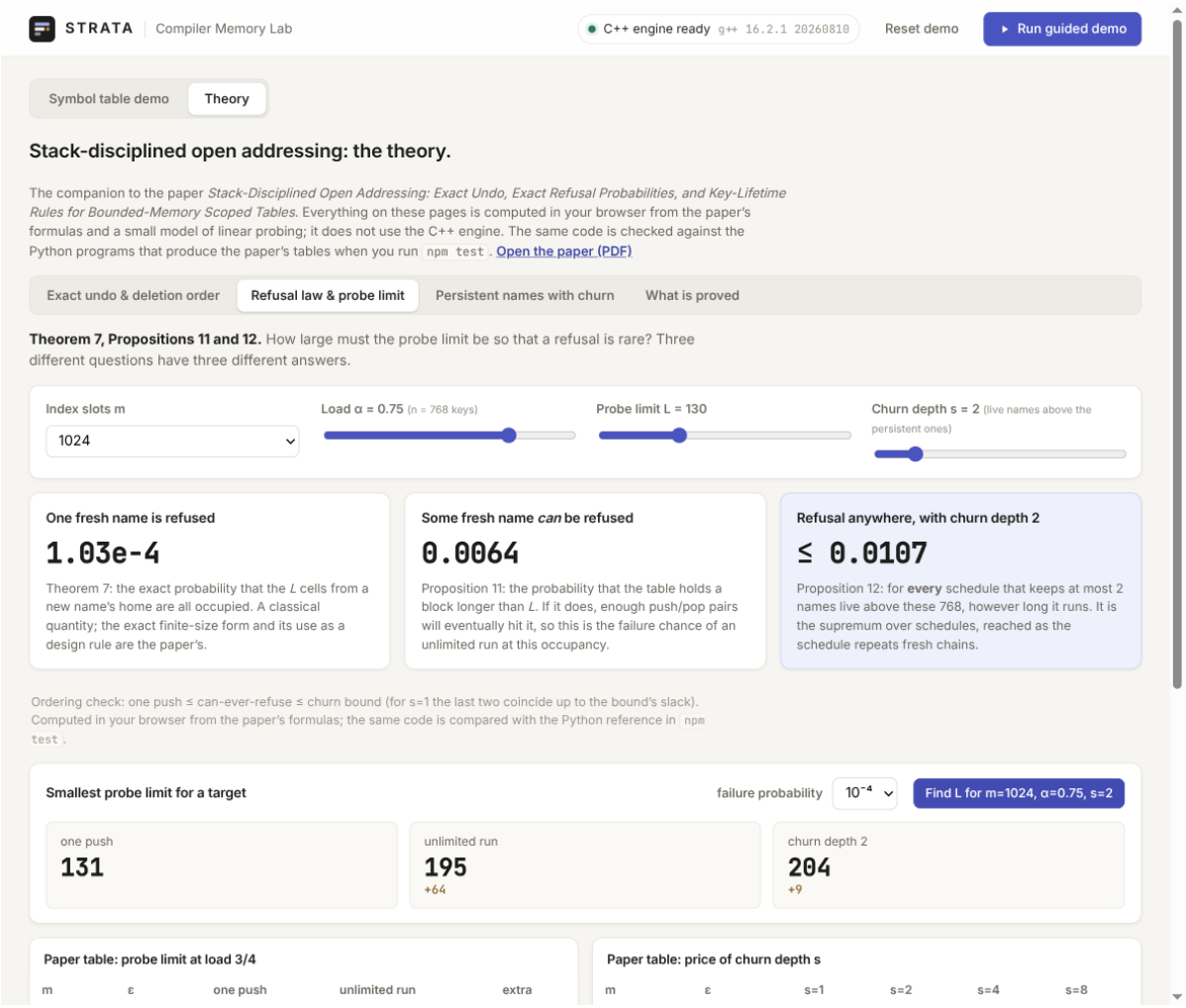


Figure 8. The interactive Theory page (React, Vite). The cards show, for $m = 1024$, load 3/4 and $L = 130$, the refusal probability of one push, the probability that some fresh name can ever be refused, and the schedule-free bound for churn depth 2; the lower panel finds the smallest probe limit for a target of 10^{-4} (131, 195 and 204).

4.2 Parameter setup

Table 2. Parameters of the experiments and of the exact computations.

Parameter	Values	Used for
Index slots m	2^8, 2^{10}, 2^{12}, 2^{14}, 2^{16} and $m \to \infty$	exact law, design tables
Load $\alpha = n/m$	3/8, 1/2, 5/8, 3/4 (0.25–0.9 for the orphan law)	all results
Target failure probability ε	10^{-2}, 10^{-4}, 10^{-6}, 10^{-9}	probe limits
Probe limit L	8–600 (curves); smallest meeting ε (tables)	refusal law
Churn depth s	1, 2, 4, 8	Proposition 12
Homes	independent uniform (model); keyed HalfSipHash-1-3 (real table); murmur3 (stored simulation)	model and validation
Enumeration	every home sequence on rings with $m \le 9$ (258 triples)	exact law
Real-table trials	400 000 per setting, 7 settings	agreement of law and table
Stored simulation	10 million samples per row, 500 tables per row	empirical reference
Monte Carlo for Prop. 12	6000 trunks, $m = 1024$, $n = 768$, $L = 80$	tightness of D_s
Software	g++ (C++14), Python 3, TeX Live, Node.js 18+	all programs

4.3 Performance analysis

Eight metrics are reported. Each has a figure or a table and a short justification. All numbers are written by the programs of the repository.

Metric 1: accuracy of the exact refusal law

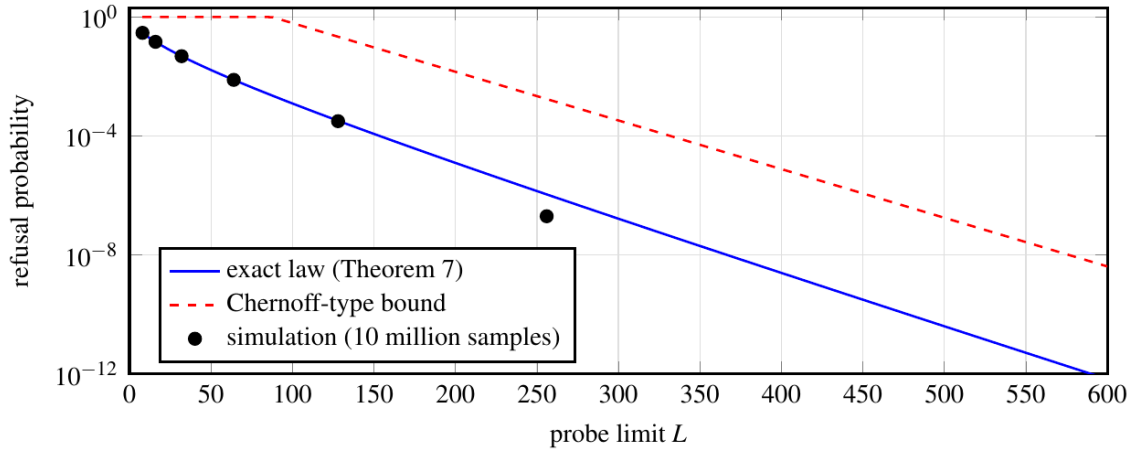


Figure 9. Probability that an insertion needs more than L probes (m = 65 536 cells, load 3/4): exact law, Chernoff-type bound, and the stored simulation (10 million samples from 500 tables). The simulated point at $L = 256$ rests on two events and is shown for completeness.

The exact law follows the simulation at every probe limit where the simulation observes enough events and lies below the Chernoff-type bound by a factor that grows with L: the uncapped bound (the plotted curve is cut at 1) is 315 times the exact value at $L = 64$ and 4637 times at $L = 512$. The point at $L = 256$ comes from two observed events in 10 million samples of 500 tables and cannot resolve a probability of the order of 10^{-6}; it is not evidence for or against the law. The next metric tests the law against the real implementation with 400 000 independent trials per setting.

Metric 2: agreement with the real table (z-score)

Table 3. Exact law against the real probe-limited table: observed frequency of a refused push in 400 000 trials, the exact value and the z-score. “Clean” trials are those in which no earlier push exceeded the limit; in them the limited table and the unlimited table stayed in lock step.

m	n	L	observed	exact	z	clean trials	mismatches
256	128	8	0.03311	0.03315	-0.13	254871	0
256	160	16	0.02643	0.02667	-0.94	307410	0
256	190	24	0.05740	0.05730	+0.26	247303	0
256	190	40	0.01372	0.01375	-0.19	369939	0
1024	760	48	0.01251	0.01286	-1.96	307286	0
1024	760	96	0.00060	0.00058	+0.52	397066	0
1024	512	24	0.00040	0.00046	-1.63	395607	0

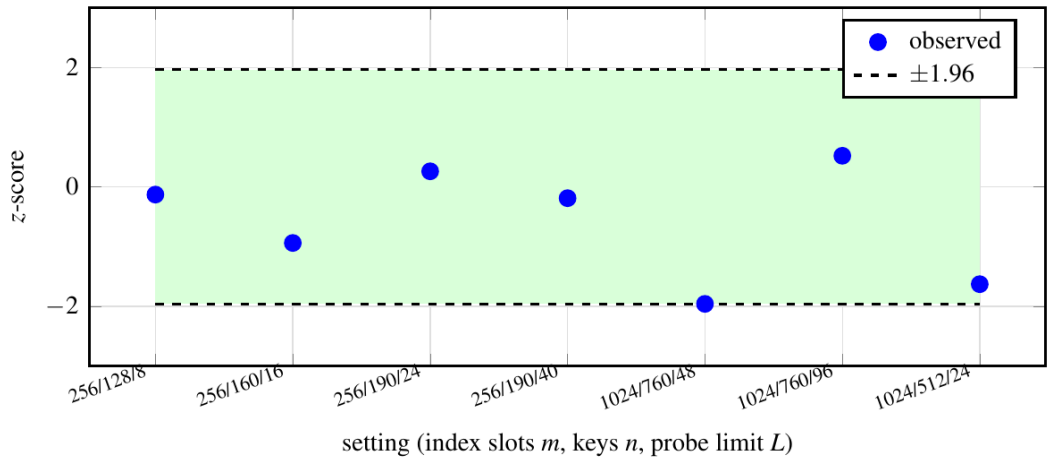


Figure 10. z-scores of the exact law against the real table. All seven settings lie inside ±1.96; the largest |z| is 1.96.

All seven z-scores lie within ±1.96, and the lock-step comparison found no mismatch. The law is therefore a statement about the implementation and not only about the model.

Metric 3: tightness of the probe limit

Table 4. Smallest probe limit L such that a push is refused with probability at most ε, from the exact law for several ring sizes and in the limit, against the Chernoff-type rule. The last column is the saving at $m = 2^{16}$.

load	ε	m=2^8	2^{10}	2^{12}	2^{14}	2^{16}	m → ∞	Chernoff	saving
3/8	10^{-6}	25	28	29	29	29	29	43	33%
3/8	10^{-9}	37	44	46	47	47	47	62	24%
1/2	10^{-6}	42	49	51	52	52	52	81	36%
1/2	10^{-9}	59	77	83	84	85	85	117	27%
5/8	10^{-6}	70	93	101	104	104	104	171	39%
5/8	10^{-9}	95	142	162	168	170	170	244	30%
3/4	10^{-6}	121	201	242	255	258	260	455	43%
3/4	10^{-9}	148	289	380	413	422	426	638	34%

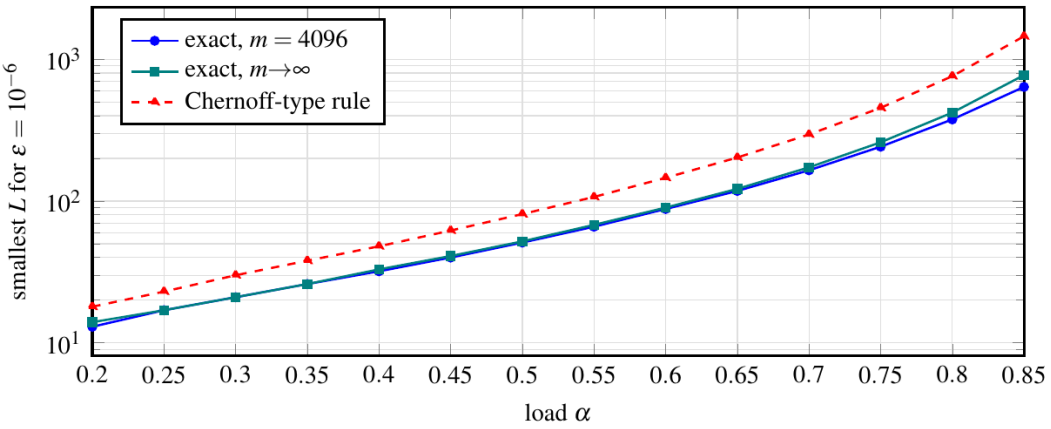


Figure 11. Smallest probe limit for ε = 10^{-6} against the load.

At load 3/4 and ε = 10^{-6} the exact limit is 258 at 2^{16} cells and 260 in the limit, against 455 for the Chernoff-type rule, a saving of 43% of the probes allowed per call; at load 3/8 the saving is 33%. The gap widens with the load because the Chernoff-type bound loses a factor that grows like $L^{3/2}$. A smaller limit means less work on a hostile input, and the exact numbers state what failure probability that limit buys.

Metric 4: probe limit for a whole fill

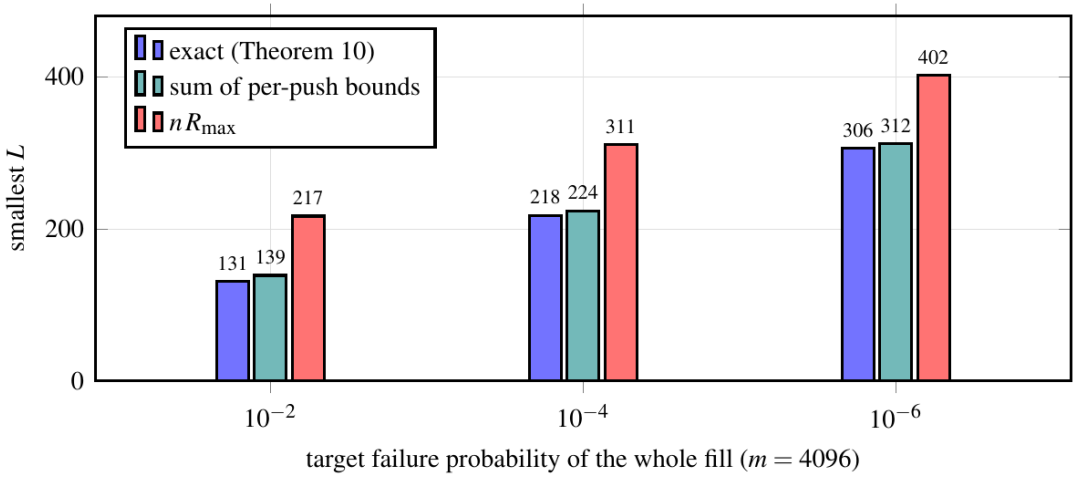


Figure 12. Smallest probe limit for a whole monotone fill to load 3/4 (m = 4096) to be refusal-free with probability at least 1 − ε′: exact (Theorem 10), the sum of the per-push bounds, and n times the maximum per-push bound.

Table 5. Whole-fill probe limits at m = 4096 and load 3/4. The last column is the true failure probability divided by the sum at the exact limit.

m	ε′	exact	sum of per-push bounds	n·R_{max}	true / sum
4096	10^{-2}	131	139	217	0.61
4096	10^{-4}	218	224	311	0.70
4096	10^{-6}	306	312	402	0.76

The exact limit is close to the one given by summing the per-push bounds and well below the limit given by n times the largest one: at ε′ = 10^{-6} it is 306 against 402, a saving of 24% in the probes allowed per call. The sum is not exact because refusals cluster: once a window of occupied cells

exists, several pushes can hit it, and the true probability is 61%–76% of the sum in the table (and up to 93% for smaller rings).

Metric 5: safety of out-of-order deletion (orphan law)

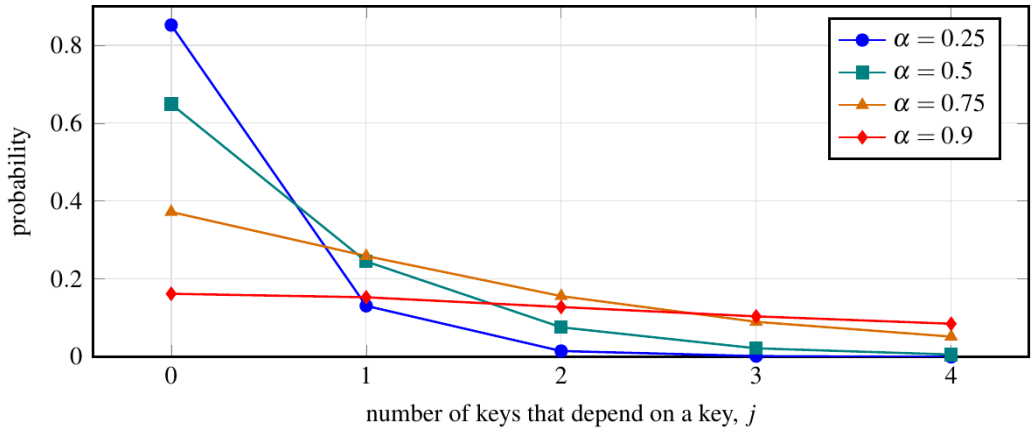


Figure 13. Probability that a key has *j* dependents, for four loads.

Table 6. Distribution of the number of keys that depend on a key (limit law) and its mean.

α	j = 0	1	2	3	4	mean
0.25	0.852	0.131	0.015	0.002	0.000	0.167
0.5	0.649	0.245	0.076	0.022	0.006	0.500
0.75	0.372	0.259	0.156	0.090	0.052	1.500
0.9	0.162	0.153	0.128	0.104	0.085	4.500

The fraction of keys that could be removed out of order without harm falls from 85% at load 1/4 to 37% at 3/4 and 16% at 9/10, and the mean number of orphans created grows as α/(2(1−α)): 0.17 at load 1/4, 1.5 at 3/4 and 4.5 at 9/10. The finite-size law was checked against enumeration of every home sequence. Stack order is the one order with no orphans at any load.

Metric 6: price of persistent names with churn

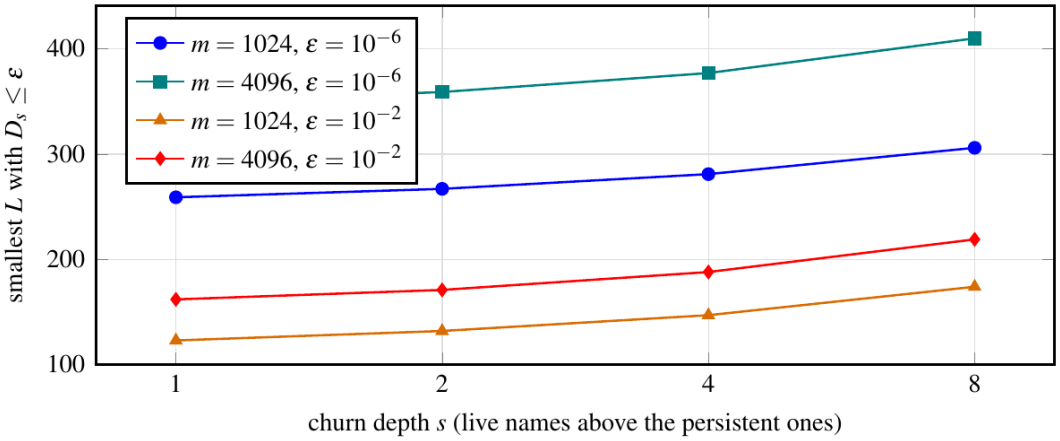


Figure 14. Smallest probe limit with $D_s \le \varepsilon$ against the churn depth *s*, at load 3/4.

Table 7. Smallest probe limit for which no schedule keeping at most *s* names above a persistent set is refused with probability above ε (Proposition 12). The column *s* = 1 is the unlimited-run limit.

m	ε	s = 1	s = 2	s = 4	s = 8
1024	10^{-2}	123	132	147	174
1024	10^{-6}	259	267	281	306
4096	10^{-2}	162	171	188	219
4096	10^{-6}	349	359	377	410

Table 8. Closeness of the closed-form bound: $m = 1024$, $n = 768$, $L = 80$; V_s estimated from 6000 simulated trunks.

s	simulated V_s	bound D_s
1	0.125	0.131
2	0.203	0.211
3	0.290	0.322
4	0.405	0.473
6	0.657	0.928

A guarantee against every schedule with eight names of churn above a persistent set costs 410 probes at $m = 4096$ and $\varepsilon = 10^{-6}$, against 349 for one level, about 17% more, and each level costs roughly eight more probes. The closed form is within 5% of the simulated vulnerability at $s = 1$ and 2, within 11% at $s = 3$ and 17% at $s = 4$, and is conservative as s grows.

Metric 7: memory overhead of the compact layout (space ledger)

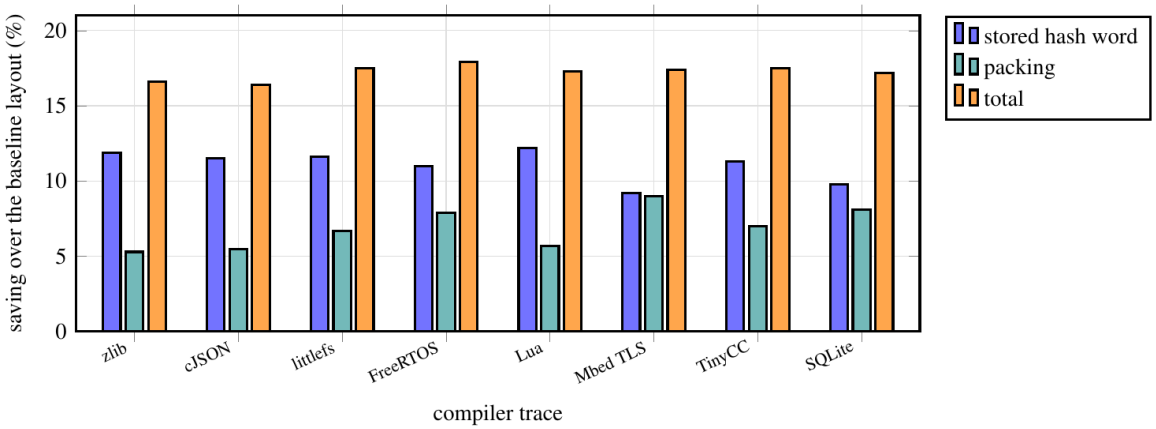


Figure 15. Saving of the compact layout over the baseline single-buffer layout on eight compiler traces, split into the stored hash word and packing.

Table 9. Smallest fixed buffer in KiB for the five layouts and the savings: dropping the stored hash ($C_0 \to C_1$), packing ($C_1 \to S$) and in total ($C_0 \to S$).

trace	C_0 (KiB)	C_1	C_2	S_raw	S	hash %	pack %	total %
zlib	18.9	16.6	16.6	16.7	15.8	11.9	5.3	16.6
cJSON	35.9	31.8	31.8	31.8	30.0	11.5	5.5	16.4
littlefs	24.2	21.4	21.4	21.5	20.0	11.6	6.7	17.5
FreeRTOS	53.6	47.8	47.8	47.8	44.0	11.0	7.9	17.9
Lua	43.4	38.1	38.1	38.1	35.9	12.2	5.7	17.3
Mbed TLS	96.9	87.9	87.9	87.9	80.0	9.2	9.0	17.4
TinyCC	189.8	168.2	168.2	168.3	156.5	11.3	7.0	17.5
SQLite	169.4	152.8	152.8	152.9	140.4	9.8	8.1	17.2

The identities reproduce all 24 measured minimum buffers of the repository exactly (to the 64-byte resolution of the measurement). The compact layout saves 16.4%–17.9% of the buffer: 9.2%–12.2% from not storing the hash and 5.3%–9.0% from packing, and interning contributes between −0.38% and 0.00%. The saving therefore comes from design choices that the ledger isolates, not from the probe limit.

Metric 8: effect of the key policy under attack

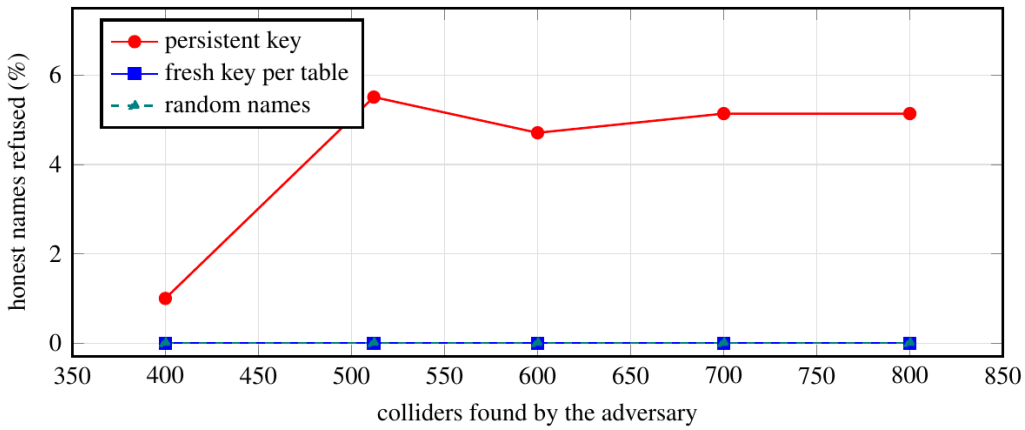


Figure 16. Fraction of honest names refused after an adversary has found colliding names (m = 2048, idealised oracle).

Table 10. Honest names refused after the attack, by key policy.

colliders	persistent key (mean)	(max)	fresh key	random names
400	1.00%	1.29%	0.00%	0.00%
512	5.51%	6.14%	0.00%	0.00%
600	4.71%	6.00%	0.00%	0.00%
700	5.14%	5.71%	0.00%	0.00%
800	5.14%	6.00%	0.00%	0.00%

With a persistent key the adversary makes 1%–6% of honest names fail; with a fresh key per table, or with random names, none does, as Proposition 13 states. The attack uses an idealised oracle: no timing was measured, and the result is a statement about the model.

Verification coverage

Table 11. Coverage of the exhaustive checks.

Check	Size	Result
Exact law against enumeration of every home sequence (exact rationals)	258 triples (m, n, L)	0 mismatches
Exact undo, five insertion rules	488 919 state comparisons	0 failures
Safe orders (Theorem 4a)	397 562 assignment–order pairs	0 failures
Exactness of safe orders and orphan law	18 settings; every assignment × every order	0 failures
Independent derivation of Theorem 7 (Abel binomial route)	481 cases	agree to $2 \cdot 10^{-10}$
Space ledger	848 449 instants of eight traces	identities hold
Proposition 12 (real insertion code)	16 920 trunks and 36 432 occupancy sets	0 failures

4.4 Comparison with related work

Three works are used for comparison. **(1) The earlier probe-limit rule.** The earlier analysis of the same table [6] and common practice set the probe limit with a Chernoff-type bound. The design table and figure of Metric 3 show that the exact law needs 24%–43% fewer probes at the same failure probability, and the real-table test shows that the exact law is the correct one. **(2) Optimal open addressing without reordering.** Farach-Colton, Krapivin and Kuszmaul [20] give insertion bounds for elastic and funnel hashing, and Krapivin, Kuszmaul and Wang [21] give interlinear probing. They bound the cost of insertions only; Theorem 6 shows that, under stack discipline, the same bounds hold for every push of every oblivious schedule, which no insertion-only analysis states. **(3) Bounded linear probing.** Alharbi, Banderier and Bouillaguet [22] analyse linear probing with at most *p* probes per insertion, for insertions in which a key that fails is discarded and redrawn; our event, refusal and stop, with deletions in stack order, is a different process, and their results do not give the refusal probabilities above.

Table 12. Comparison with the related work used for evaluation.

Aspect	Chernoff-type rule (earlier STRATA analysis)	Insertion-only bounds (FKK; KKW)	Bounded linear probing (ABB)	This work
Operations	insertions	insertions	insertions	pushes and pops in stack order
Quantity	probe limit from a tail bound	expected probe index at load $1 - \delta$	successful insertions under a probe cap	exact refusal probabilities; safe orders
Exact at finite m	no (loose by $\Theta(L^{3/2})$)	no (orders of magnitude)	asymptotic and finite-size results	yes, and validated on the real table
Deletions	not treated	not treated	not treated	exact undo; only stack order is safe for every table
Probe limit	455 (load 3/4, 10^{-6})	not a design rule	not a design rule	258 (load 3/4, 10^{-6}, $m = 2^{16}$)

Table 13. Expected probe index at load $1 - \delta$ (orders of magnitude, $\delta \to 0$). Under stack discipline the worst push equals the average over the worst oblivious schedule (Theorem 6). Linear and uniform probing: Knuth [7] and Yao [39]; funnel and elastic hashing: [20]; interlinear probing: [21] (upper bounds).

scheme	insertion-only: worst / amortised	stack discipline: worst = amortised
linear probing	δ^{-2} / δ^{-1}	δ^{-2}
uniform probing	δ^{-1} / $\log \delta^{-1}$	δ^{-1}
funnel hashing (greedy)	$\log^2 \delta^{-1}$ / $\log \delta^{-1}$	$\log^2 \delta^{-1}$
elastic hashing (non-greedy)	$\log \delta^{-1}$ / 1	$\log \delta^{-1}$
interlinear probing (stable)	$\delta^{-1} \log \delta^{-1}$ / δ^{-1}	$\delta^{-1} \log \delta^{-1}$

5. Conclusion and scope for future work

- **The proposed work.** We asked what the last-in-first-out discipline of scopes gives an open-addressing table that must live in one fixed buffer, and answered with exact statements: popping the newest key is an exact undo, stack order is the only universally safe order, and the refusal probability of a probe-limited table is exactly computable.

- **Existing methodology.** Analyses of open addressing cover insertions, deletions are known to be difficult, and probe limits were set with Chernoff-type bounds that are loose by a factor growing like $L^{3/2}$.
- **Proposed methodology and its advantage.** Reducing a push/pop workload to an insertion-only table (Theorem 2) transfers every insertion-only result; the exact laws replace loose bounds, and the schedule-free bound covers a persistent set of names with bounded churn for the whole life of the table.
- **Tools.** Exact rational arithmetic, exhaustive enumeration, the real C++ table and an independent derivation were used, and one script regenerates every number, table and figure.
- **Results.** At load 3/4 and failure probability 10^{-6} the exact probe limit is 258 against 455 for the Chernoff-type rule; the whole-fill limit is 306 against 402; guarding against any schedule with eight names of churn costs 410 against 349 for one level. The law agrees with the real table (all $|z| \le 1.96$, no mismatch), and 16.4%–17.9% of the buffer is saved by the compact layout.
- **Scope for future work.** A limit law for refusal probabilities on wide and deep scope trees is open: simulations show that failures cluster on shared prefixes, a mixture structure for which we found no clean statistic. An unconditional version of the probe-complexity separation needs a lower bound for stable schemes under stack discipline. Compilers that keep several scopes alive at once, a measured timing channel for the key results, and a machine-checked version of the proofs are further directions.

6. References

[1] Gurdeep Singh, Robbert and Christophe Scholliers. (2019) “WARDuino: a dynamic WebAssembly virtual machine for programming microcontrollers.” Proceedings of the 16th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes (MPLR 2019): 27–36. doi:10.1145/3357390.3361029.

[2] Rouvoet, Arjen, Hendrik van Antwerpen, Casper Bach Poulsen, Robbert Krebbers, and Eelco Visser. (2020) “Knowing when to ask: sound scheduling of name resolution in type checkers derived from declarative specifications.” Proceedings of the ACM on Programming Languages (OOPSLA) 4 (OOPSLA): 1–28 (Crossref gives no article number). doi:10.1145/3428248.

[3] van Antwerpen, Hendrik and Eelco Visser. (2021) “Scope States: Guarding Safety of Name Resolution in Parallel Type Checkers.” 35th European Conference on Object-Oriented Programming (ECOOP 2021), LIPIcs vol. 194 194 (1 (article number)): 1:1–1:29. doi:10.4230/LIPIcs.ECOOP.2021.1.

[4] Appel, Andrew W. (1998) “Modern Compiler Implementation in C.” Cambridge University Press. doi:10.1017/CBO9781139174930.

[5] Graham, S. L., W. N. Joy, and O. Roubine. (1979) “Hashed Symbol Tables for Languages With Explicit Scope Control.” Proc. SIGPLAN Symposium on Compiler Construction: 50–57. doi:10.1145/800229.806953.

[6] Singh, Shubhi, and Adhyan Jain. (2026) “STRATA: a budgeted, stack-disciplined scoped symbol table.” Public repository cdesproject (https://github.com/adhyan-jain/cdesproject); the directory theory/ holds this article's sources and verification programs.

[7] Knuth, Donald E. (1998) “The Art of Computer Programming, Volume 3: Sorting and Searching.” Addison-Wesley.

[8] Janson, Svante and Alfredo Viola. (2016) “A Unified Approach to Linear Probing Hashing with Buckets.” Algorithmica 75 (4): 724–781. doi:10.1007/s00453-015-0111-x.

[9] Bender, Michael A., Alex Conway, Martin Farach-Colton, William Kuszmaul, and Guido Tagliavini. (2023) “Iceberg Hashing: Optimizing Many Hash-Table Criteria at Once.” Journal of the ACM 70 (6): 1–51. doi:10.1145/3625817.

[10] Bender, Michael A., Martin Farach-Colton, John Kuszmaul, William Kuszmaul, and Mingmou Liu. (2022) “On the optimal time/space tradeoff for hash tables.” Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC '22): 1284–1297. doi:10.1145/3519935.3519969.

[11] Dalal, Ketan, Luc Devroye, and Ebrahim Malalla. (2023) “Two-Way Linear Probing Revisited.” Algorithms 16 (11): 500. doi:10.3390/a16110500.

[12] Hu, Yang, William Kuszmaul, Jingxun Liang, Stefan Walzer, Huacheng Yu, and Renfei Zhou. (2026) “Quadratic Probing Revisited: Smoothed Analysis and the Fall of Robin Hood.” arXiv preprint (cs.DS): arXiv:2607.13247. https://arxiv.org/abs/2607.13247.

[13] Kenyon, Richard and Mei Yin. (2023) “Parking Functions: From Combinatorics to Probability.” Methodology and Computing in Applied Probability 25 (1): Article 32. doi:10.1007/s11009-023-10022-5.

[14] Aguilar-Fraga, Tomas, Jennifer Elder, Rebecca E. Garcia, et al. (2024) “Interval and l-interval rational parking functions.” Discrete Mathematics & Theoretical Computer Science 26:1 (Permutation Patterns 2023): #10 (article id 12598). doi:10.46298/dmtcs.12598.

[15] Colaric, Emma, Ryan DeMuse, Jeremy L. Martin, and Mei Yin. (2021) “Interval parking functions.” Advances in Applied Mathematics 123: 102129. doi:10.1016/j.aam.2020.102129.

[16] Csikor, Levente, Dinil Mon Divakaran, Min Suk Kang, et al. (2019) “Tuple Space Explosion: A Denial-of-Service Attack against a Software Packet Classifier.” Proceedings of the 15th International Conference on Emerging Networking Experiments and Technologies (CoNEXT '19): 292–304. doi:10.1145/3359989.3365431.

[17] He, Le and Hongbo Yu. (2023) “Cryptanalysis of Reduced-Round SipHash.” The Computer Journal 67 (3): 875–883. doi:10.1093/comjnl/bxad026.

[18] Bender, Michael A., Bradley C. Kuszmaul, and William Kuszmaul. (2021) “Linear Probing Revisited: Tombstones Mark the Demise of Primary Clustering.” 2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS): 1171–1182. doi:10.1109/FOCS52979.2021.00115.

[19] Braverman, Mark and William Kuszmaul. (2024) “Tight Analyses of Ordered and Unordered Linear Probing.” 2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS): 606–635. doi:10.1109/FOCS61266.2024.00046.

[20] Farach-Colton, Martin, Andrew Krapivin, and William Kuszmaul. (2024) “Optimal Bounds for Open Addressing Without Reordering.” 2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS): 594–605. doi:10.1109/FOCS61266.2024.00045.

[21] Krapivin, Andrew, William Kuszmaul, and Yixuan Wang. (2026) “Linear Probing with Non-Greedy Insertions.” arXiv preprint (cs.DS): arXiv:2607.17494. https://arxiv.org/abs/2607.17494.

[22] Alharbi, Ahmed, Cyril Banderier, and Charles Bouillaguet. (2026) “Bounded Linear Probing Hashing.” 37th International Conference on Probabilistic, Combinatorial and Asymptotic Methods for the Analysis of Algorithms (AofA 2026), LIPIcs vol. 381 381: 28:1–28:17. doi:10.4230/LIPIcs.AofA.2026.28.

[23] Bender, Michael A., William Kuszmaul, and Renfei Zhou. (2025) “Optimal Non-oblivious Open Addressing.” Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC '25): 268–277. doi:10.1145/3717823.3718215.

[24] Bender, Michael A., Alex Conway, Martin Farach-Colton, William Kuszmaul, and Guido Tagliavini. (2023) “Tiny Pointers.” Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA): 477–508. doi:10.1137/1.9781611977554.ch21.

[25] van Antwerpen, Hendrik, Casper Bach Poulsen, Arjen Rouvoet, and Eelco Visser. (2018) “Scopes as types.” Proceedings of the ACM on Programming Languages (OOPSLA) 2 (OOPSLA): 1–30 (Crossref gives no article number). doi:10.1145/3276484.

[26] Zwaan, Aron, Hendrik van Antwerpen, and Eelco Visser. (2022) “Incremental type-checking for free: using scope graphs to derive incremental type-checkers.” Proceedings of the ACM on Programming Languages (OOPSLA2) 6 (OOPSLA2): 424–448. doi:10.1145/3563303.

[27] Ball, Thomas, Peli de Halleux, and Michał Moskal. (2019) “Static TypeScript: an implementation of a static compiler for the TypeScript language.” Proceedings of the 16th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes (MPLR 2019): 105–116. doi:10.1145/3357390.3361032.

[28] Mochizuki, Fumika, Tetsuro Yamazaki, and Shigeru Chiba. (2025) “BlueScript: A Disaggregated Virtual Machine for Microcontrollers.” The Art, Science, and Engineering of Programming 10 (3): Article 21. doi:10.22152/programming-journal.org/2025/10/21.

[29] Naor, Moni and Eylon Yogev. (2019) “Bloom Filters in Adversarial Environments.” ACM Transactions on Algorithms 15 (3): 1–30. doi:10.1145/3306193.

[30] Clayton, David, Christopher Patton, and Thomas Shrimpton. (2019) “Probabilistic Data Structures in Adversarial Environments.” Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19): 1317–1334. doi:10.1145/3319535.3354235.

[31] Blair, William, Andrea Mambretti, Sajjad Arshad, et al. (2020) “HotFuzz: Discovering Algorithmic Denial-of-Service Vulnerabilities Through Guided Micro-Fuzzing.” Proceedings 2020 Network and Distributed System Security Symposium (NDSS). doi:10.14722/ndss.2020.24415.

[32] Janson, Svante. (2005) “Individual Displacements for Linear Probing Hashing With Different Insertion Policies.” ACM Transactions on Algorithms 1 (2): 177–213. doi:10.1145/1103963.1103964.

[33] Sanders, Peter. (2018) “Hashing with Linear Probing and Referential Integrity.” arXiv preprint arXiv:1808.04602.

[34] Pittel, B. (1987) “Linear Probing: The Probable Largest Search Time Grows Logarithmically With the Number of Records.” Journal of Algorithms 8 (2): 236–249. doi:10.1016/0196-6774(87)90040-X.

[35] Petersson, Niclas. (2009) “The Maximum Displacement for Linear Probing Hashing.” PhD thesis, Uppsala University.

[36] ISO/IEC (International Organization for Standardization / International Electrotechnical Commission). (2014) “ISO/IEC 14882:2014, Information technology - Programming languages - C++ (Edition 4).” ISO/IEC standard, published December 2014. https://www.iso.org/standard/64029.html. Accessed October 2026.

[37] Meta Platforms, Inc. and React contributors. (n.d.) “React (react.dev documentation website).” Website. https://react.dev/. Accessed October 2026.

[38] VoidZero Inc. and Vite contributors. (n.d.) “Getting Started | Vite.” Website (Vite documentation). https://vite.dev/guide/. Accessed October 2026.

[39] Yao, Andrew Chi-Chih. (1985) “Uniform hashing is optimal.” Journal of the ACM 32 (3): 687–693. doi:10.1145/3828.3836.

Author Contributions

Shubhi Singh designed and implemented STRATA, its tests and benchmarks and wrote the earlier empirical study this work extends. Adhyan Jain directed the theoretical work: the adversarial review of the earlier study, the statements and proofs of the theory, the verification programs and the interactive page. Both authors are responsible for the content. Contribution roles follow the repository record (docs/CONTRIBUTIONS.md and docs/AI_USE_DISCLOSURE.md).

Acknowledgements

The authors thank the maintainers of the open-source tools used: GCC, Python, TeX Live, LibreOffice, React and Vite.

Funding

This research received no external funding.

Competing interests

The authors declare no competing interests.

Use of AI assistance

The software, tests, benchmarks and the earlier empirical manuscript that this article extends were produced by an AI coding assistant working in the environment of the first author, under the authors' direction, as recorded in the repository. This article was developed with the same kind of tool: Anthropic's Claude (model Sonnet 5.5, through the Claude Code command-line tool) was used, at one author's direction, to review the earlier work adversarially, to propose and derive the theorems and proofs, to write the verification programs, to build the interactive page and to draft the text. Every number in the article is produced by those programs, which can be re-run. The tool is not an author; the listed authors are responsible for the content. The proofs are pen-and-paper and were checked by exhaustive and statistical programs, not by a proof assistant.

Data and code availability

All sources, programs, traces and raw results are in the public repository [6]; the directory `theory/` holds the full manuscript with every proof, the verification programs and the interactive page, and `bash theory/verify/run_all.sh` regenerates every number, table and figure of this article.